

10 - Trabajando con archivos

Programación I
Ingeniería Electrónica y Telecomunicaciones

Comisión 3
Profesor: Lautaro Linquimán
Universidad Nacional de Río Negro



Repaso express de la clase anterior 06/05

- ¿Que es un diccionario?
- ¿Que es una tupla?
- ¿Que es un conjunto?

Manejo de archivos

¿Por qué son importantes los archivos?

Hasta ahora, los programas que escribimos “olvidan” todo cuando terminan: las variables viven solo mientras el programa está corriendo.

Los archivos permiten **guardar información** para volver a usarla después: nombres, notas, productos, ventas, configuraciones o resultados.

En sistemas reales esto es clave: un banco no puede olvidarse tu saldo, una red social no puede perder tus publicaciones y una aplicación no puede borrar tus datos cada vez que se cierra.



Todos los lenguajes de programación manejan de alguna manera archivos.

LECTURA DE ARCHIVOS

Leyendo nuestro primer archivo

```
# Crear archivo datos.txt  
Hola Mundo Soy Un Archivo  
Que Existe En el FileSystem  
Si me cambian y me guardan  
Y ejecutan el siguiente codigo  
Me verán en la pantalla
```

```
# Crear lector_archivos.py  
archivo = open("datos.txt", "r")  
contenido = archivo.read()  
archivo.close()  
  
print(contenido)
```

Mecanismo de lectura de archivos

Python ofrece múltiples mecanismos para la lectura de archivos.

- Volcar el archivo completo
- Volcar linea a linea
- Volcar todas las líneas a una lista

Para leer archivos usamos:

```
open( NOMBRE_ARCHIVO, "r")
```

La "r" de READ

```
# Leyendo el archivo por completo #
archivo = open("datos.txt", "r")
# Lee el archivo completo y lo vuelca a un string.
contenido = archivo.read()
# Usamos end="" para evitar el salto de línea que agrega print
print(contenido, end="")
archivo.close()
#-----

# Leyendo linea a linea #
archivo = open("datos.txt", "r")
# Lee el archivo hasta encontrar una linea completa delimitada por \n
linea = archivo.readline()
print(linea.strip())

archivo.close()

# Volcando el archivo a una lista #
archivo = open("datos.txt", "r")

# Lee el archivo y lo vuelca linea a linea en una lista
lineas = archivo.readlines()
print(lineas)

for linea in lineas:
    print(linea.strip())

archivo.close()
```

Puntero de archivo: A tener en cuenta

Una vez que leemos un archivo dentro del programa usando **read()**, **readline()** o **readlines()**, el contenido leído no vuelve a estar disponible automáticamente para una nueva lectura desde el mismo punto inicial.

Esto ocurre porque Python maneja la lectura del archivo mediante un puntero interno o cursor de lectura. Cada vez que ejecutamos una función de lectura, ese puntero avanza. Por eso, si volvemos a llamar a **read()**, **readline()** o **readlines()**, la lectura continuará desde la posición actual del puntero, no desde el inicio del archivo.

Para volver a leer desde el principio, debemos mover el puntero manualmente con **seek(0)**.

```
archivo = open("datos.txt", "r")
contenido = archivo.read()
contenido_sin_nada = archivo.read()
print("Contenido: ", contenido)
print("Contenido vacio: ", contenido_sin_nada)
archivo.close()
```

```
archivo = open("datos.txt", "r")
contenido = archivo.read()
archivo.seek(0)
contenido_sorpresa = archivo.read()
print("Contenido: ", contenido)
print("Contenido sorpresa: ", contenido_sorpresa)
archivo.close()
```

Operando los datos de un archivo

Podemos pasar de un archivo de texto plano a una estructura de datos dentro de nuestro programa.

```
# numeros.txt
```

```
1
```

```
2
```

```
3
```

```
4
```

```
# Mostrador de numeros.py
```

```
numeros = []
```

```
archivo = open("numeros.txt", "r")
```

```
for numero in archivo.readlines():
```

```
    numeros.append(int(numero))
```

```
archivo.close()
```

```
print(sum(numeros))
```

Operando los datos de un archivo

Podemos pasar de un archivo de texto plano a una estructura de datos dentro de nuestro programa.

```
# personas.csv
nombre,apellido
juan, carlos
pedro, martinez
carlos, juan martinez

# lector de base de datos
personas = []
archivo = open("personas.csv", "r")
for persona in archivo.readlines()[1:]:
    nombre, apellido = persona.strip().split(",")
    personas.append({
        "nombre": nombre,
        "apellido": apellido
    })

archivo.close()

print(personas)
```

Ejercicio de lectura de archivos

Ejercicio 1: Crear el archivo nombres.txt y escribir 5 nombres.

Leer el archivo desde nuestro programa y crear una lista con los 5 nombres e imprimir en pantalla mostrando su índice.

Ejercicio 2: Crear el archivo productos.csv y escribir 5 líneas el siguiente formato: nombre_producto; precio_producto; stock;

Leer el archivo desde nuestro programa y crear un diccionario con todos los productos.

Mostrar en pantalla ordenadamente.

Escribiendo
archivos

Mecanismo de escritura de archivos

Python ofrece dos mecanismos para la escritura de archivos.

- write: en formato texto
- writelines: en formato línea

Para escribir archivos usamos:

```
open( NOMBRE_ARCHIVO, "w")
```

La "w" de WRITE

OJO: el uso de write va a pisar el contenido de nuestro archivo cada vez que se abra.

```
# Desde un interprete

# Creamos el archivo
archivo = open("mis_datos.txt", "w")

# Escribimos en el archivo
archivo.write("Hola 1\n")
archivo.write("Hola 2\n")
archivo.write("Hola 3")

# Cerramos el archivo
# Creamos el archivo
archivo = open("mis_datos.txt", "w")

datos = ["Hola 1\n", "Hola 2\n", "Hola 3\n", "Hola 4"]

# Escribimos en el archivo
# archivo.writelines("\n".join(datos)) # Esto al final da un
string
archivo.writelines(datos)
# "\n".join nos permite unir listas de texto con "\n"

# Cerramos el archivo
archivo.close()
```

\n

El carácter de control `\n` es muy importante cuando trabajamos con archivos.

`\n` representa el salto de línea.

Sirve para tener una visual más limpia del archivo, pero también para delimitar su lectura programática línea a línea.

Para que python entienda que hay agregar un salto de línea es necesario ponerlo explícitamente.

```
archivo = open("mis_datos.txt", "w")
archivo.write("Hola 1")
archivo.write("Hola 2")
archivo.write("Hola 3")
archivo.close()

# Archivo de salida: Hola 1Hola 2Hola 3
```

```
archivo = open("mis_datos.txt", "w")
archivo.write("Hola 1\n")
archivo.write("Hola 2\n")
archivo.write("Hola 3")
archivo.close()

# Archivo de salida:
# Hola 1
# Hola 2
# Hola 3
```

Ejercicio de escritura de archivos

Ejercicio 3: Crear un programa que solicite 5 nombres al usuario.

Luego escribir los 5 nombres en un archivo de texto, cada nombre en una línea.

Ejercicio 4: Crear un programa que solicite al usuario 5 nombre y su documento.

Luego escribir datos con el siguiente formato en cada línea: nombre, documento.

El archivo se debe llamar registros.csv

Más modos de manejo de archivos



Tablita de modos

Modo	Nombre	¿Permite leer?	¿Permite escribir?	¿Qué pasa si el archivo no existe?	¿Qué pasa si el archivo ya existe?
"r"	Solo lectura	Sí	No	Error	Se abre sin modificarlo
"r+"	Lectura y escritura	Sí	Sí	Error	Se abre sin borrar su contenido
"w"	Solo escritura	No	Sí	Se crea	Se borra su contenido anterior
"w+"	Escritura y lectura	Sí	Sí	Se crea	Se borra su contenido anterior
"a"	Solo añadir	No	Sí	Se crea	Agrega contenido al final
"a+"	Añadir y leer	Sí	Sí	Se crea	Agrega contenido al final

Ejemplos

```
# "a": agrega al final.
```

```
archivo = open("demo.txt", "a")
```

```
archivo.write("Nueva linea\n")
```

```
archivo.close()
```

```
# "r+": lee y escribe. No borra el contenido.
```

```
archivo = open("demo.txt", "r+")
```

```
contenido = archivo.read()
```

```
archivo.write("Agregado con r+\n")
```

```
archivo.close()
```

```
# "w+": borra, escribe y permite leer.
```

```
archivo = open("demo.txt", "w+")
```

```
archivo.write("Texto nuevo\n")
```

```
archivo.seek(0)
```

```
print(archivo.read())
```

```
archivo.close()
```

```
# "a+": agrega al final y permite leer.
```

```
archivo = open("demo.txt", "a+")
```

```
archivo.write("Agregado con a+\n")
```

```
archivo.seek(0)
```

```
print(archivo.read())
```

```
archivo.close()
```

Documentación: <https://docs.python.org/es/3.14/library/functions.html#open>

Estructura de archivos



Organizando el caos

Para evitar que nuestro programa se convierta en un caos de archivos, podemos usar un recurso muy simple: **carpetas**.

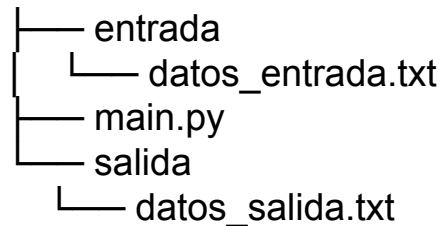
Por ejemplo, al mismo nivel que nuestro archivo de código, podemos crear una carpeta **entrada/** para los archivos que el programa lee y una carpeta **salida/** para los archivos que el programa genera.

Cuando los archivos están dentro de carpetas, ya no alcanza con escribir solo el nombre del archivo. Tenemos que indicar el camino para llegar hasta él.

A ese camino lo llamamos **ruta relativa**.

Una ruta relativa indica, desde la carpeta donde estamos trabajando, qué carpetas (una o más) hay que recorrer y cuál es el nombre del archivo que queremos leer o escribir.

estructura-de-archivos



Basados en la estructura anterior si queremos leer `datos_entrada.txt` debemos definirlo de la siguiente manera: `open("entrada/datos_entrada.txt", "r")`

Para `datos_salida.txt`:
`open("salida/datos_salida.txt", "w")`

Ejercicio de estructura de archivos

EJ5: Crear la estructura de archivos de la derecha.

Completar frases.txt a gusto.

En [main.py](#) escribir un programa que escriba el contenido de entrada/frase.txt en la carpeta salida con el nombre frase_copia.txt

```
copiador/  
├── main.py  
├── entrada/  
│   └── frase.txt  
└── salida/
```

EJ6: Crear la estructura de archivos de la derecha.

Completar el archivo notas a gusto.

Escribir un programa que lea el archivo notas.txt y lo imprima linea a linea.

```
lector_de_notas/  
├── main.py  
└── datos/  
    ├── alumnos/  
    └── notas.txt
```

Ejercicio integrador

Ei1 Parte1: Escribir un programa que lea el siguiente archivo y extraiga a una lista de diccionarios nombre, notas y promedio.

Archivo:

Ana,8-3-6
Juan,5-8-4
Pedro,9-10-9
Lucia,4-6-5

Usar la estructura de archivos propuesto en los ejercicios anteriores.

Ayudita: usar `split("-")` para separar las notas.

Parte 2: A partir de la estructura de datos creada escribir en otro archivo con el siguiente formato en cada línea.

NOMBRE, PROMEDIO, APRUEBA O NO APRUEBA

Se aprueba con 7.

PDFS y TAREAS



Todo el material en <https://github.com/Ymil/unrn-programacion-1/>

FIN

Volvemos el miércoles a las 17